

تمرین هفتم

برنامه نویسی پیشرفته

دکتر ملکی مجد

۲۹ اردیبهشت ۱۴۰۴

زمستان - بهار ۴۰۳۲



دانشگاه علم و صنعت ایران

دانشگاه علم و صنعت ایران

نکات مهم تمرین

- تنها زبان برنامه‌نویسی مجاز برای این تمرین C# است.
- از پاسخ تحویل داده شده تمرین ها ارائه گرفته خواهد شد.
- در صورت مشاهده هرگونه کپی برداری هم کپی کننده و هم کپی شونده نمره صفر خواهند گرفت.
- در صورت مشاهده استفاده از هرگونه LLM برای پاسخ گویی به سوالات به جای شما، نمره شما صفر خواهد شد.
- برای هر سری تمرین سه روز مهلت تحویل با تاخیر دارید و مجموعاً ده روز مهلت ارسال با تاخیر دارید.

طراحان تمرین

در صورت بروز هرگونه مشکل یا سوالی در مورد تمرین، می‌توانید با طراحان تمرین تماس بگیرید.

• سوال اول - سید علی حسینی @sseeeyeedd_ir

• سوال دوم - محمد یآوری @EmmWhy

سوالات

LINQ - سوال اول

موضوع تمرین پردازش برخی از داده های مربوط به اطلاعاتی از شرکت های مختلف از کشورهای دنیا است. فایل csv.organizations شامل داده های مربوط به شرکت ها است. پردازش اولیه در کد پایین پیاده سازی شده است. کاری که شما باید انجام دهید این است که Method Extension های مناسبی را پیاده سازی کنید و از آنها برای پاسخگویی به سوالاتی که در زیر مطرح شده است، استفاده کنید.

```
using System.Text;

namespace HW7
{
    public class Organization
    {
        public Organization(string line)
        {
            var toks = line.Split(',');
            index = toks[0];
            Name = toks[1];
            Website = toks[2];
            Country = toks[3];
            Description = toks[4];
            Founded = int.Parse(toks[5]);
            Industry = toks[6];
            NumberOfEmployees = int.Parse(toks[7]);
        }
        public string index;
        public string Name;
        public string Website;
        public string Country;
        public string Description;
        public int Founded;
        public string Industry;
        public int NumberOfEmployees;
    }

    internal class Question1
    {
        static void Main(string[] args)
        {
            var data = File.ReadAllLines(@"..\..\..\organizations.csv")
                .Skip(1)
                .Select(line => new Organization(line));
            Console.WriteLine($"Question 1: {data.ExtensionMethodPlaceholder1()}");
            Console.WriteLine($"Question 2: {data.ExtensionMethodPlaceholder2()}");
            Console.WriteLine($"Question 3: {data.ExtensionMethodPlaceholder3()}");
            Console.WriteLine($"Question 4: {data.ExtensionMethodPlaceholder4()}");
            Console.WriteLine($"Question 5: {data.ExtensionMethodPlaceholder5()}");
            Console.WriteLine($"Question 6: {data.ExtensionMethodPlaceholder6()}");
            Console.WriteLine($"Question 7: {data.ExtensionMethodPlaceholder7()}");
            Console.WriteLine($"Question 8: {data.ExtensionMethodPlaceholder8()}");
            Console.WriteLine($"Question 9: {data.ExtensionMethodPlaceholder9()}");
            Console.WriteLine($"Question 10: {data.ExtensionMethodPlaceholder10()}");
        }
    }
}
```

```

public static class Extensions
{
    public static string ExtensionMethodPlaceHolder1(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder2(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder3(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder4(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder5(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder6(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder7(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder8(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder9(this IEnumerable<Organization> list)
    {
    }
    public static string ExtensionMethodPlaceHolder10(this IEnumerable<Organization> list)
    {
    }
}
}

```

۱. تعداد شرکت هایی که سایت آنها دارای دامنه .net است.
۲. متداول ترین صنعت در شرکت های ایرانی
۳. میانگین تعداد کارمندان شرکت هایی که در زمینه صنایع شیمیایی (Chemicals) کار می کنند و پس از سال ۲۰۰۰ تاسیس شده اند.
۴. جمع تعداد کارمندان شرکت های فعال در زمینه E-Learning یا Online Publishing
۵. نام شرکت اسپانیایی با بیشترین تعداد کارمند.
۶. نام کشوری که بیشترین تعداد تاسیس شرکت پس از ۲۰۱۰ را داشته است.
۷. نام و سال تأسیس ۵ شرکت قدیمی تر را که در صنعت Software فعالیت می کنند، نمایش دهید.
۸. نام ۱۰ شرکت در صنعت Design و ۱۰ شرکت در صنعت Fine Art به ترتیب براساس کمترین تعداد کارمند و قدیمی ترین سال تاسیس
۹. نام ۱۰ تا از جدیدترین شرکت هایی که در توضیحات آنها کلمه circuit – چه lowercase چه uppercase – وجود دارد به ترتیب تعداد کارمند
۱۰. وبسایت ۱۰ شرکت اول براساس جدیدترین سال تاسیس که یا در صنعت کشتی سازی (Fishery) فعالیت دارند یا متعلق به کشور ایتالیا هستند. (در این مورد اجازه ی استفاده از || در where را ندارید. یک روش جدیدی پیدا کنید).

انبار داری هوشمند:

کلاس های مورد نیاز:

```
public abstract class WarehouseSection<T>

    protected List<T> items
    public void AddItem(T item)
    public void RemoveItem(Func<T, bool> predicate)
    public T? GetItem(Func<T, bool> predicate)
    public List<T> FilterItems(Func<T, bool> predicate)
    public bool Exists(Func<T, bool> predicate)
    public void UpdateItem(int id, T newItem)
    public void ClearAll()
    public int Count()
    public void SortItems(Comparison<T> comparison)
    public void ListItems()
    protected abstract void DisplayItem(T item);
    public abstract string CategoryName();
    public virtual List<T> FilterByCategoryProperty(string key)
```

- Items: برای نگهداری آیتم ها
- AddItem: یک آیتم به لیست آیتم ها اضافه می کند.
- RemoveItem: آیتم هایی را که با شرط داده شده مطابقت دارند حذف می کند.
- GetItem: اولین آیتمی که با شرط داده شده مطابقت دارد را بر میگرداند یا اگر آیتمی یافت نشد مقدار null بر می گرداند.
- FilterItems: لیستی از آیتم هایی که با شرط داده شده مطابقت دارند را بر میگرداند.
- Exists: اگر آیتمی وجود دارد که با شرط داده شده مطابقت داشته باشد true و در غیر این صورت false بر میگرداند.
- UpdateItem: آیتم با آیدی داده شده را با آیتم جدید جایگزین می کند.
- ClearAll: همه آیتم ها را پاک می کند.
- Count: تعداد آیتم های موجود را ذخیره می کند.
- SortItems: آیتم ها را بر اساس معیار داده شده مرتب می کند.
- ListItems: با استفاده از متود نمایش آیتم همه آیتم ها را لیست می کند.
- DisplayItem: متودی که بازنویسی خواهد شد و کار آن نمایش اطلاعات یک آیتم است.

- CategoryName: در کلاس های فرزند مقدار خواهد گرفت.
- FilterByCattegoryProperty: به طور پیشفرض لیست خالی بر میگردداند و در کلاس های فرزند بازنویسی می شود.

```
public class WarehouseCategory<T> : WarehouseSection<T>

    public string Name { get; private set; }
    public WarehouseCategory(string name) // Constructor
    public override string CategoryName() // to simply return the category name
    protected override void DisplayItem(T item) // default DisplayItem
```

- Name: نام category
- DisplayItem: به طور پیشفرض اطلاعات مشترک بین انواع آیتم هارا چاپ می کند.

```
public class Electronic

    public int ID { get; set; }
    public string Name { get; set; }
    public string Brand { get; set; }
    public Electronic(int id, string name, string brand)
    public override string ToString()
```

بازنویسی متود ToString برای استفاده در متود DisplayItem انجام می شود.

```
public class ElectronicsWarehouse : WarehouseCategory<Electronic>
{
    public ElectronicsWarehouse() : base( ... )
    protected override void DisplayItem(Electronic item)
    public List<Electronic> GetByBrand(string brand)
    public override List<Electronic> FilterByCategoryProperty(string key)
}
```

دو متود `DisplayItem` و `FilterByCategoryProperty` باز نویسی می شوند. (فیلتر بر اساس نام برند به عنوان کلید انجام می شود)

```
public class Furniture
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Material { get; set; }
    public Furniture(int id, string name, string material)
    public override string ToString()
}
```

مشابه کلاس `Electronic`

```
public class FurnitureWarehouse : WarehouseCategory<Furniture>
{
    public FurnitureWarehouse() : base( ... )
    protected override void DisplayItem(Furniture item)
    public List<Furniture> GetByMaterial(string material)
    public override List<Furniture> FilterByCategoryProperty(string key)
}
```

مشابه کلاس `ElectronicWarehouse` با این تفاوت که فیلتر بر اساس جنس انجام میشود.

```

public class Book
{
    public int ID { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public Book(int id, string title, string author)
    public override string ToString()
}

```

مشابه کلاس Furniture و Electronic

```

public class BookWarehouse : WarehouseCategory<Book>
{
    public BookWarehouse() : base( ... )
    protected override void DisplayItem(Book item)
    public List<Book> GetByAuthor(string author)
    public override List<Book> FilterByCategoryProperty(string key)
}

```

مشابه کلاس FurnitureWarehouse و ElectronicWarehouse با این تفاوت که فیلتر بر اساس نام نویسنده انجام می شود.

نکته: پیاده سازی منو لازم نیست و تنها کفایست متود های پیاده سازی شده را در main تست کنید.